

Lab 7: Implementació pro2::Vector

Explicació de la solució

1. L'estratègia de memòria dinàmica

Per implementar un `Vector`, necessitem gestionar la memòria manualment al **heap**. La clau és separar el concepte de **mida** (`size_`, quants elements hi ha) i **capacitat** (`capacity_`, quant espai hem reservat).

El mètode `reallocate_`

És el cor de la classe. S'encarrega de demanar un nou bloc de memòria, copiar els elements antics i alliberar el bloc vell.

```
void reallocate_(int new_capacity) {
    T* new_data = new T[new_capacity];
    int to_copy = (new_capacity < size_) ? new_capacity : size_;
    for (int i = 0; i < to_copy; ++i) {
        new_data[i] = data_[i];
    }
    delete[] data_; // Alliberem el bloc vell
    data_ = new_data;
    capacity_ = new_capacity;
    size_ = to_copy;
}
```

2. La Regla dels Tres

Com que la nostra classe gestiona recursos (memòria dinàmica), hem de definir: 1. **Destructor**: Per evitar fugues de memòria (`delete[] data_`). 2. **Constructor de còpia**: Per fer còpies profundes. Si no el féssim, dos vectors apuntarien a la mateixa memòria. 3. **Operador d'assignació**: Similar a la còpia, però ha de netejar la memòria prèvia i vigilar amb l'auto-assignació (`this != &other`).

3. Gestió de la Capacitat

push_back: Cost Amortitzat $\mathcal{O}(1)$

Quan el vector s'omple, **doblem** la capacitat. Això fa que les operacions de còpia (costoses) siguin cada vegada més esporàdiques.

pop_back: Evitant el Thrashing

Si reduïssim la memòria just quan el vector està a la meitat, una seqüència de `push/pop` alternats seria molt lenta. Per això esperem que estigui a $1/4$ de la capacitat per reduir-lo a la **meitat**.

Solució Completa: `vector.hh`

Aquí tens la implementació completa que passa tots els tests del laboratori:

```

“cpp [vector.hh] #ifndef VECTOR_HH #define VECTOR_HH
#include “assert.hh” #include
namespace pro2 {
template class Vector { T* data_; int size_; int capacity_;
static constexpr int initial_capacity_ = 4;

void reallocate_(int new_capacity) {
    T* new_data = new T[new_capacity];
    int to_copy = (new_capacity < size_) ? new_capacity : size_;
    for (int i = 0; i < to_copy; ++i) {
        new_data[i] = data_[i];
    }
    delete[] data_;
    data_ = new_data;
    capacity_ = new_capacity;
    size_ = to_copy;
}

public: using iterator = T; using const_iterator = const T;
// Constructors i Destructor
Vector() : data_(nullptr), size_(0), capacity_(0) {}

Vector(int n) {
    assert(n >= 0, "Vector::Vector(int): mida negativa!");
    data_ = new T[n];
    size_ = n;
    capacity_ = n;
}

Vector(int n, const T& x) {
    assert(n >= 0, "Vector::Vector(int, const T&): mida negativa!");
    data_ = new T[n];
    size_ = n;
    capacity_ = n;
    for (int i = 0; i < n; ++i) data_[i] = x;
}

Vector(const Vector& other) : data_(new T[other.capacity_]), size_(other.size_), capacity_(other.capacity_) {
    for (int i = 0; i < size_; ++i) data_[i] = other.data_[i];
}

~Vector() {
    delete[] data_;
}

Vector& operator=(const Vector& other) {
    if (this != &other) {
        delete[] data_;
        size_ = other.size_;
        capacity_ = other.capacity_;
        data_ = new T[capacity_];
        for (int i = 0; i < size_; ++i) data_[i] = other.data_[i];
    }
}

```

```

    }
    return *this;
}

// Accés
T& operator[](int i) {
    assert(i >= 0 && i < size_, "Vector::operator[]: índex fora de rang!");
    return data_[i];
}

const T& operator[](int i) const {
    assert(i >= 0 && i < size_, "Vector::operator[]: índex fora de rang!");
    return data_[i];
}

// Capacitat
int size() const { return size_; }
int capacity() const { return capacity_; }
bool empty() const { return size_ == 0; }

void clear() {
    delete[] data_;
    data_ = nullptr;
    size_ = 0;
    capacity_ = 0;
}

void reserve(int n) {
    assert(n >= 0, "Vector::reserve: capacitat negativa!");
    if (n > capacity_) reallocate_(n);
}

void resize(int n) {
    assert(n >= 0, "Vector::resize: mida negativa!");
    reallocate_(n);
    size_ = n;
}

// Modificació
void push_back(const T& x) {
    if (size_ == capacity_) {
        reallocate_(capacity_ == 0 ? initial_capacity_ : 2 * capacity_);
    }
    data_[size_] = x;
    ++size_;
}

void pop_back() {
    assert(size_ > 0, "Vector::pop_back: vector buit!");
    --size_;
    if (size_ < capacity_ / 4) {
        reallocate_(capacity_ / 2);
    }
}

```

```
// Iteradors
iterator begin() { return data_; }
const_iterator begin() const { return data_; }
iterator end() { return data_ + size_; }
const_iterator end() const { return data_ + size_; }

};

} // namespace pro2
#endif ““
```